Problem Set 02 — Inside the Lineage

Corresponds to: 02-ontogeny — The C compiler family tree: DMR → PCC → GCC → TCC.

This problem set goes inside the code. You will read specific functions, trace specific paths, and make one small modification. Line numbers given are for the original source; they may drift slightly in your practice copy.

Allow yourself three to four hours across multiple sessions.

Problem 1 — The thin wrapper [reading, 10 min]

Open `practice/tcc/tcc.c`. It is 432 lines. This is not where most of TCC lives.

Read lines 22–29. Find the `ONE_SOURCE` macro. What is it doing? Now find the `#include` statements in tcc.c. Which source files are pulled in when `ONE_SOURCE` is set to 1?

Find `main()` at line 289. Read it. What are the first three calls main() makes before it gets to file compilation?

> **Write down:**
> • Which files are included via `ONE_SOURCE`?
> • What does `main()` do before it compiles anything?
> • Why might Bellard have chosen a single-compilation-unit approach?

Problem 2 — The real core [reading, 15 min]

Open `practice/tcc/tccgen.c`. Run:

```
wc -l tccgen.c
```

This file — not tcc.c — is the heart of TCC. It contains the parser, the type checker, and the code generator, deeply interleaved.

Find the function declaration for `block()` near line 132. This function parses a single C statement. Read its signature and the comment above it if any.

Now search for `TOK_IF` in tccgen.c:

```
grep -n "TOK_IF" tccgen.c
```

Find the case that handles an `if` statement (around line 7194). Read it without trying to understand every instruction. Get the shape of it.

> **Write down:** how many lines does the `TOK_IF` case occupy? What is the first function it calls after recognizing `if`?

Problem 3 — Follow an if [reading, 20 min]

Stay in the `TOK_IF` case in `tccgen.c`. Read it end to end — it handles the full if/else construct in one place.

> **Write down:**
> How does TCC represent a conditional branch? What function emits the actual branch instruction?
> What is `gjmp` and what does it do? (Search for its definition in tccgen.c.)
> When TCC sees an `else` clause, what does it do with the jump that would skip the else body?

You don't need to understand the x86 instructions being emitted. Describe the control flow structure in terms of: *recognize*, *emit branch*, *compile body*, *patch address*.

Problem 4 — The architecture question [analysis, 20 min]

In PCC (the Portable C Compiler, 1977), the parser and the code generator are strictly separated: the front end builds a representation, the back end walks it. In TCC, they are interleaved: the function that recognizes `if` immediately emits branch instructions.

Locate two functions in `tccgen.c` where this interleaving is most visible — where the same lines both parse a syntactic construct and emit machine code for it.

> **Write down:**
> • The two functions and their approximate line numbers
> • One advantage of interleaving: when would it be faster or simpler?
> • One disadvantage: what does it make harder?
> • What would you have to change in TCC to separate the front and back ends?

Problem 5 — Build and self-compile [hands-on, 30 min]

TCC can compile itself. Do this in two steps.

Step 1: Build TCC from source using the system compiler:

```
cd practice/tcc
./configure && make clean && make
./tcc --version    # note the output
```

Step 2: Use TCC to compile TCC:

```
./tcc -o tcc2 tcc.c -DONE_SOURCE
./tcc2 --version   # should match
```

Note: this simplified command works for most configurations. If it fails, check the Makefile for a `cross` target or read `Makefile` for the full self-compilation command.

Write down:

• Did both versions report the same version string?

• How would you verify that `tcc` and `tcc2` are truly equivalent? (What inputs could you test?)

• What is Thompson's "trusting trust" argument, applied to this situation? (One sentence.)

Problem 6 — Modification: banner [hands-on, 20 min]

Open `practice/tcc/tcc.c`. Find where TCC prints its version string — search for the `--version` output in the code:

```
grep -n "version\|TCC\|Tiny" tcc.c tcctools.c
```

Add a line to the version output that includes your name and the year. Something like:

```
Course edition — [Your Name], 2026
```

Rebuild:

```
make clean && make
./tcc --version
```

Your addition should appear. Now run `tcc -run scratch/hello.c` to verify the compiler still works after your modification.

> **Write down:** exactly what you changed and where. What did you learn about how the version string is constructed?

Problem 7 — GCC entry point [reading, 15 min]

In `02/gcc/` (the original — do not modify), find the main compiler driver. GCC is large; use search:

```
grep -rn "^int main" 02/gcc/gcc/ | head -10
```

Find `gcc.c` or the equivalent driver file. Read `main()`. How does GCC's entry point compare to TCC's? Count the lines in GCC's main() function.

> **Write down:**
> • Where is GCC's main() relative to tcc's?
> • What does GCC's main() do that TCC's does not?
> • What does GCC call after parsing arguments that corresponds to TCC's compile loop?

Problem 8 — RTL [research, 20 min]

GCC uses Register Transfer Language (RTL) as its intermediate representation between the parsed source and the machine code. TCC uses no intermediate representation — it emits machine code directly.

Find the GCC Internals Manual at `gnu.org/software/gcc/` and read the section titled "RTL Representation" (or search the PDF for "rtl").

> **Write down:**
> • What is an RTL expression? Give the general form.
> • Give one concrete example of an RTL expression and explain what C operation it represents.
> • How does having an IR enable optimization passes that TCC's architecture cannot support?

Next: Problem Set 03 — history, geographies, and the long line. Open `ps-03-phylogeny.pdf`.